

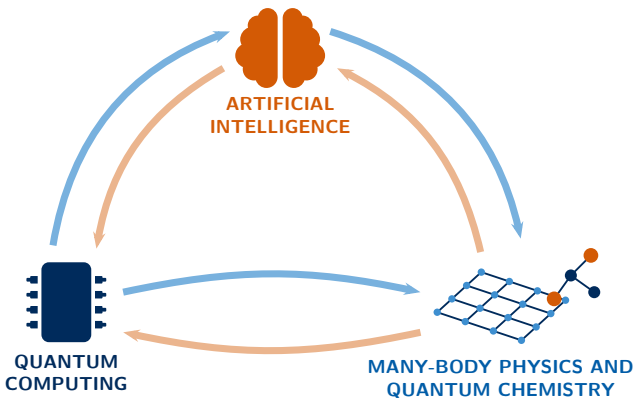
Lecture 1: Artificial Intelligence Research
ML vs DL
Neural Networks Anatomy

Deep Learning in Classical and Quantum Physics

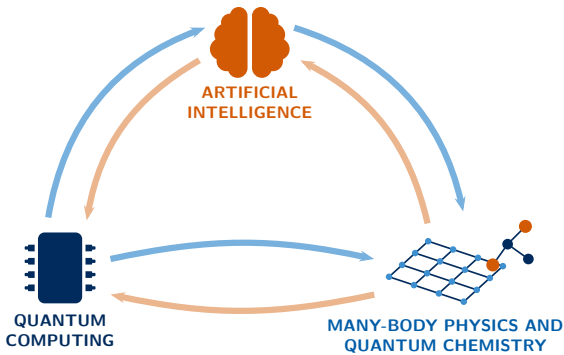
Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

Big Goal in Quantum Physics



Big Goal in Quantum Physics



Examples:

1. Efficient quantum circuit compilations
2. Discoveries of new phases of matter
3. Optimal control of quantum systems
4. Quantum State/Process Tomography
5. Neural Network Quantum states ansatzes
6. ...

1

Artificial Intelligence Research



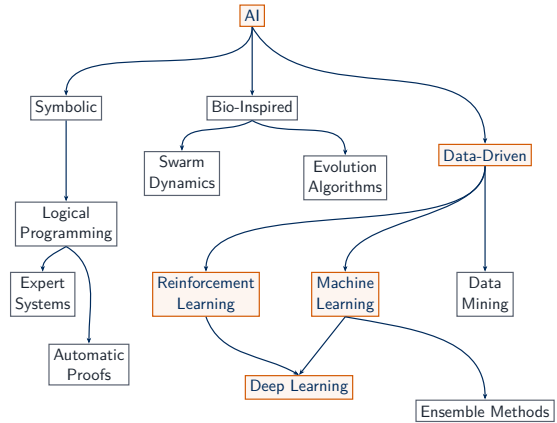
JAGIELLONIAN UNIVERSITY
IN KRAKOW

Artificial Intelligence Research

1. Can we simulate "intelligent" behavior?
2. Can we solve problem not by "hard coded" approach, but much more like "intelligent humans"?

Many approaches:

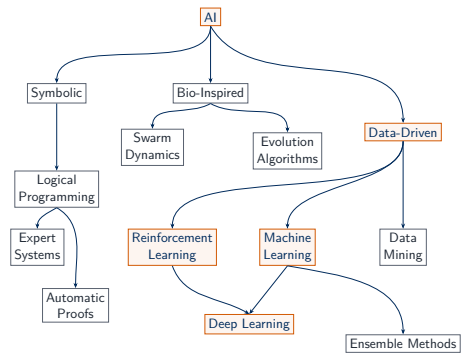
1. Symbolic / axiomatic
2. Rule-based : logical programming
3. Expert systems : knowledge based
4. Bio-inspired mechanisms
5. **Data-Driven**



Artificial Intelligence Research

Early Foundations (1940s–1950s)

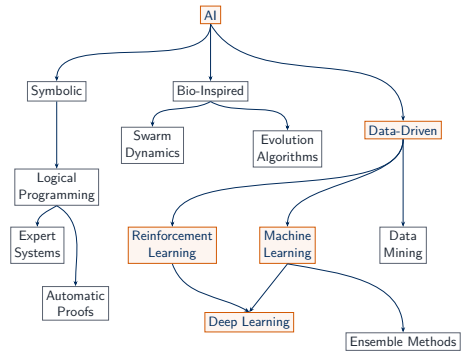
- **1943**: Warren McCulloch and Walter Pitts publish *A Logical Calculus of Ideas Immanent in Nervous Activity*, proposing the first mathematical model of a neural network.
- **1950**: Alan Turing publishes *Computing Machinery and Intelligence*, introducing the Turing Test and discussing machine intelligence.
- **1956**: The Dartmouth Conference, organized by John McCarthy and others, is held, where the term *Artificial Intelligence* is coined, marking the formal beginning of AI research.



Artificial Intelligence Research

Early Progress (1950s–1960s)

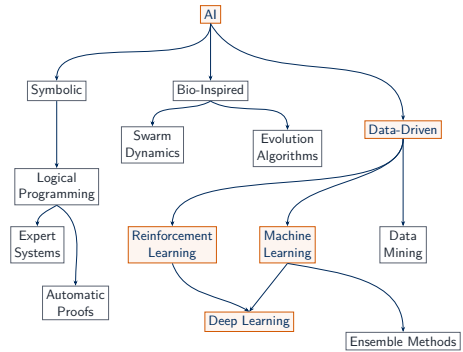
- **1956**: Allen Newell and Herbert A. Simon develop the Logic Theorist, considered the first AI program.
- **1958**: John McCarthy develops the Lisp programming language, which becomes a primary language for AI research.
- **1966**: Joseph Weizenbaum creates ELIZA, an early natural language processing program simulating conversation.
- **1969**: Marvin Minsky and Seymour Papert publish *Perceptrons*, analyzing the capabilities and limitations of neural networks, particularly in handling non-linear problems.



Artificial Intelligence Research

Expert Systems Era (1970s–1980s)

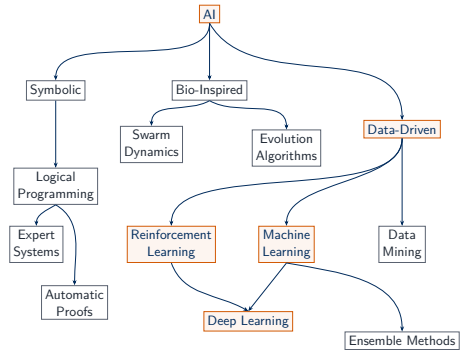
- **1972**: PROLOG, a programming language based on logic, is developed and becomes a key tool in AI research.
- **1979**: The Stanford Cart, an early autonomous vehicle, successfully crosses a room avoiding obstacles, demonstrating early robotics and computer vision capabilities.
- **1980**: The rise of expert systems, such as MYCIN for medical diagnosis, showcases practical applications of AI.
- **1987–1993**: The *AI Winter* occurs due to reduced funding and interest, as expectations for AI progress are not met.



Artificial Intelligence Research

Machine Learning Renaissance (1990s–2000s)

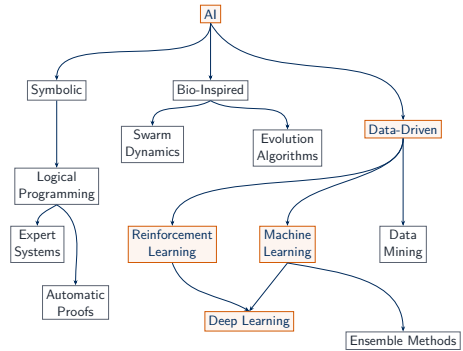
- **1997**: IBM's Deep Blue defeats world chess champion Garry Kasparov, showcasing the potential of AI in strategic games.
- **1998**: Yann LeCun and colleagues develop LeNet-5, a convolutional neural network for handwritten digit recognition, advancing image processing.
- **2000**: Cynthia Breazeal develops Kismet, a robot capable of recognizing and simulating emotions, advancing human-robot interaction.
- **2006**: Geoffrey Hinton and colleagues revive deep learning through the development of deep belief networks, sparking renewed interest in neural networks.



Artificial Intelligence Research

Modern Deep Learning Era (2010s–Present)

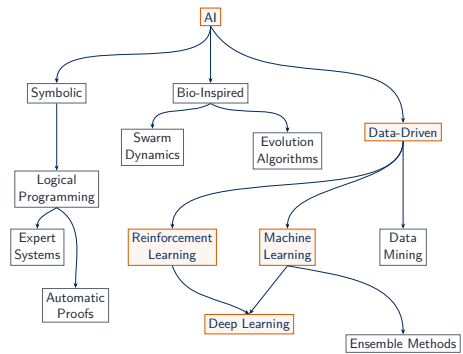
- **2012**: AlexNet, developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, wins the ImageNet competition, revolutionizing computer vision with deep convolutional neural networks.
- **2014**: Ian Goodfellow and colleagues introduce Generative Adversarial Networks (GANs), enabling AI to generate realistic data.
- **2015**: Google DeepMind's AlphaGo defeats professional Go player Fan Hui, marking a significant achievement in AI.
- **2016**: AlphaGo defeats Lee Sedol, one of the world's best Go players, in a landmark victory.



Artificial Intelligence Research

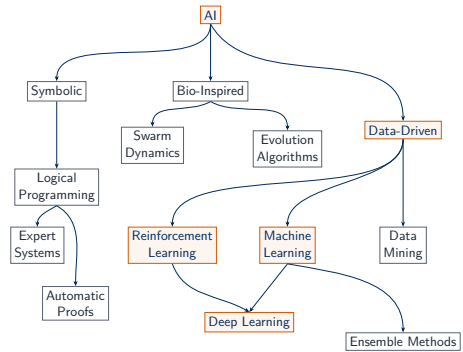
Modern Deep Learning Era (2010s–Present)

- **2017**: The Transformer model is introduced by Vaswani et al., leading to breakthroughs in natural language processing (e.g., BERT, GPT series).
- **2020**: OpenAI releases GPT-3, setting a new standard for large language models and applications.
- **2020**: DeepMind's AlphaFold AI system demonstrates the ability to predict protein structures with remarkable accuracy, solving a 50-year-old grand challenge in biology.

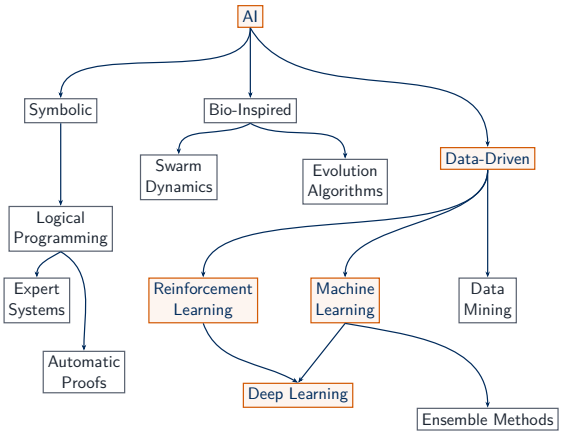


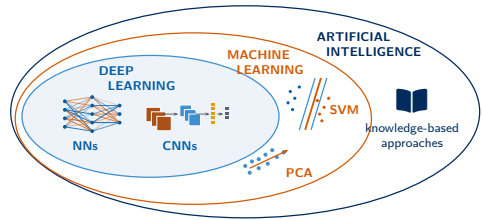
Artificial Intelligence Research

- **2022**: OpenAI's ChatGPT showcases advanced conversational AI capabilities to a broad audience.
- **2024**: Demis Hassabis and John Jumper of DeepMind, along with David Baker, are awarded the Nobel Prize in Chemistry for their work on protein structure prediction using AI.
- **2024**: Geoffrey Hinton and John Hopfield receive the Nobel Prize in Physics for their foundational contributions to artificial neural networks and machine learning.

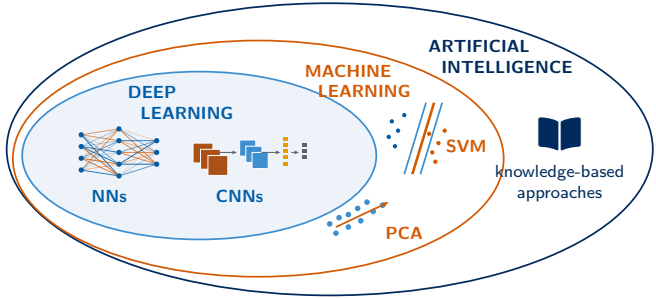


Hierarchy of AI



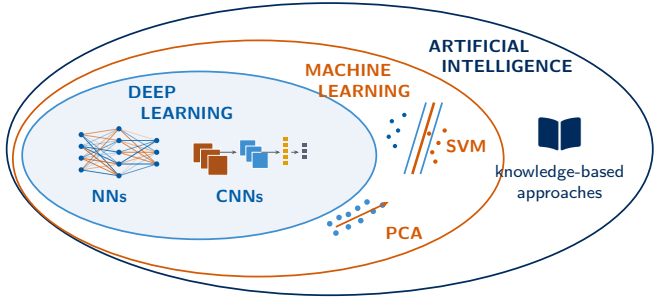


Traditional vs Data-Driven Programming





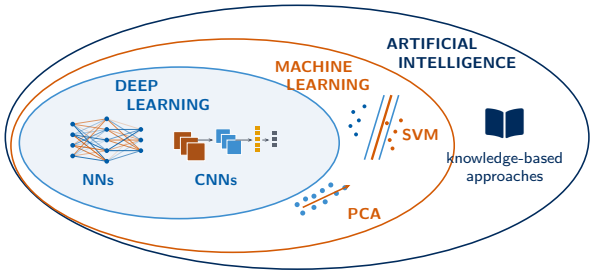
Traditional vs Data-Driven Programming







Machine Learning – Knowledge-Based Programming



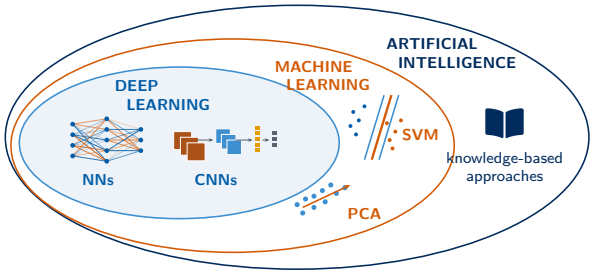


Main ML paradigms

1. Unsupervised Learning
2. Supervised Learning

Teacher-Student scheme

Machine Learning – Knowledge-Based Programming



Main ML paradigms

1. Unsupervised Learning
2. Supervised Learning

Teacher-Student scheme

3. **Reinforcement Learning**

   Finding best strategy

2

ML vs DL: Why, What and How



JAGIELLONIAN UNIVERSITY
IN KRAKOW

The Difference Between Two Dog Breeds





German Shepherd





Dachshund

Algorithm- vs Data-Driven Software Development







German Shepherd







Dachshund



Algorithm-driven

Quite difficult to hard-code algorithm for image classification

Algorithm- vs Data-Driven Software Development





German Shepherd





Dachshund

Let's provide many examples of images and extract critical features of different breeds



Data-driven

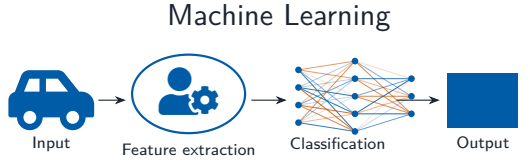
ML vs DL: Feature Extraction








German Shepherd Dachshund



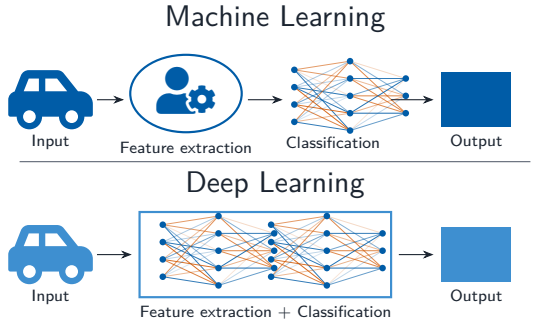
ML vs DL: Feature Extraction







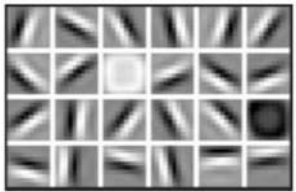

German Shepherd Dachshund



https://www.xenonstack.com/blog/static/public/uploads/media/machine-learning-vs-deep-learning.png

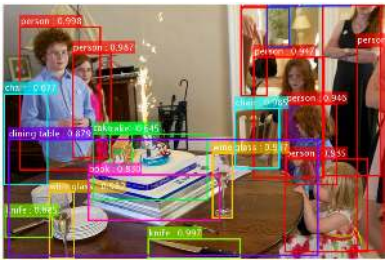
Crucial features are extracted automatically

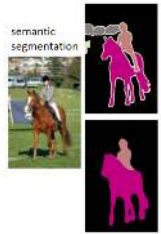
Classification, Detection, Segmentation is Based on Extracted Features

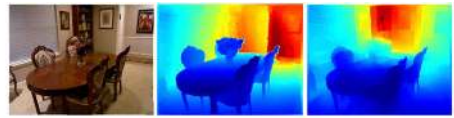














3

Neural Networks Anatomy

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Neural Networks Anatomy

Neural Networks are non-linear,
parametrized functions with nested structure

$$f_{\vec{\theta}} : \mathbf{x} \in \mathbb{R}^m \rightarrow \mathbf{y} \in \mathbb{R}^n, \;\; m, n \in \mathbb{N}$$

Neural Networks Anatomy

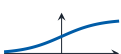
Non-linear, parametrised functions:
Layers: input $\rightarrow$ hidden $\rightarrow$ output

Each neuron computes:
$y = \sigma\Big(\sum_i x_i w_i + b\Big)$

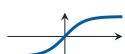
Layers are nested:
$f_{\vec{\theta}}(\mathbf{x}) = h^{(L)} \circ \cdots \circ h^{(1)}(\mathbf{x})$

$\sigma =$ **activation function**, e.g.:

sigmoid: $\frac{1}{1+e^{-x}}$



tanh: $\frac{e^x - e^{-x}}{e^x + e^{-x}}$

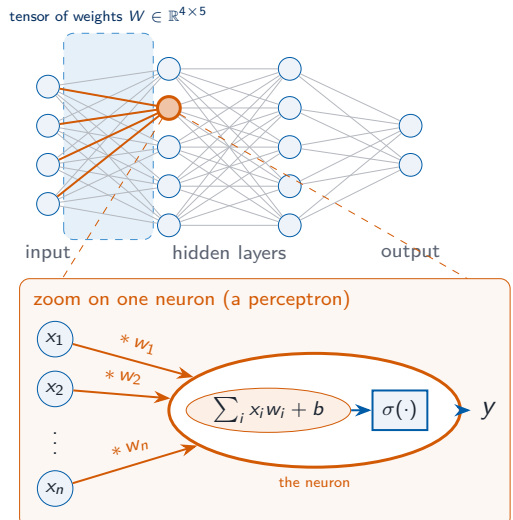


ReLU: $\max(0, x)$

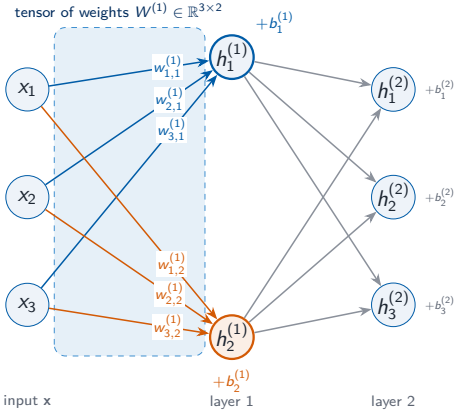


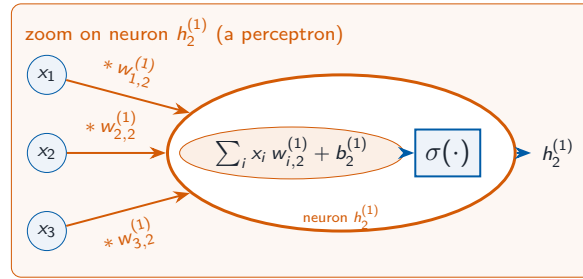
step: 1 if $x > 0$, else 0





Neural Networks Anatomy

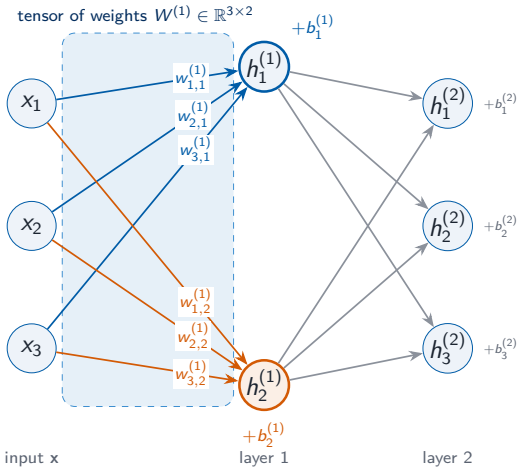




$W \in \mathbb{R}^{N\times M}$: N nodes on the left, M on the right

$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}, \qquad h_2^{(1)} = \sigma\left(z_2^{(1)}\right)$$

Neural Networks Anatomy



Each neuron adds up what arrives along its edges:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$

$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Same pattern for every neuron j — a sum over the inputs i:

$$z_j^{(1)} = \sum_{i=1}^{3} x_i \, w_{i,j}^{(1)} + b_j^{(1)}, \qquad h_j^{(1)} = \sigma\left(z_j^{(1)}\right)$$

Neural Networks Anatomy

The two neurons, term by term:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$
$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Matrix product, by definition

row vector $\mathbf{x}$ times matrix W:

$$(\mathbf{x}\,W)_j = \sum_i x_i\, W_{i,j}$$

= row $\mathbf{x}$ "times" column j of W

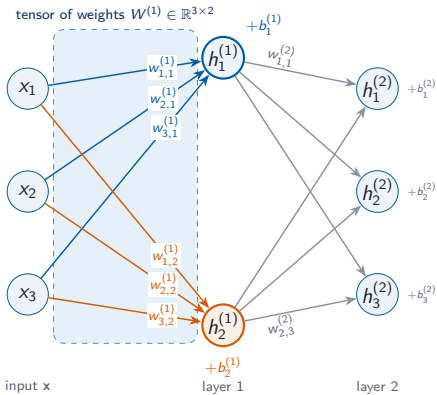
This is exactly the sum each neuron computes. All neurons at once:

$$\begin{bmatrix} z_1^{(1)} & z_2^{(1)} \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} & b_2^{(1)} \end{bmatrix}$$

$$\boxed{\mathbf{z}^{(1)} = \mathbf{x}\,W^{(1)} + \mathbf{b}^{(1)}}, \qquad \mathbf{h}^{(1)} = \sigma\big(\mathbf{z}^{(1)}\big) \text{ element by element}$$

column j of $W^{(1)}$ = the incoming edges of neuron j; shape (inputs $\times$ neurons) $= 3 \times 2$

Neural Networks Anatomy



Layer 2 does the same, with $\mathbf{h}^{(1)}$ as its input:

$$h_k^{(2)} = \sigma\Big(\sum_{j=1}^{2} h_j^{(1)}\, w_{j,k}^{(2)} + b_k^{(2)}\Big)$$

$$\iff \quad \mathbf{h}^{(2)} = \sigma\big(\mathbf{h}^{(1)} W^{(2)} + \mathbf{b}^{(2)}\big)$$

$W^{(2)}$: 2×3 (2 inputs, 3 neurons)

Any layer l $\quad (\mathbf{h}^{(0)} = \mathbf{x})$

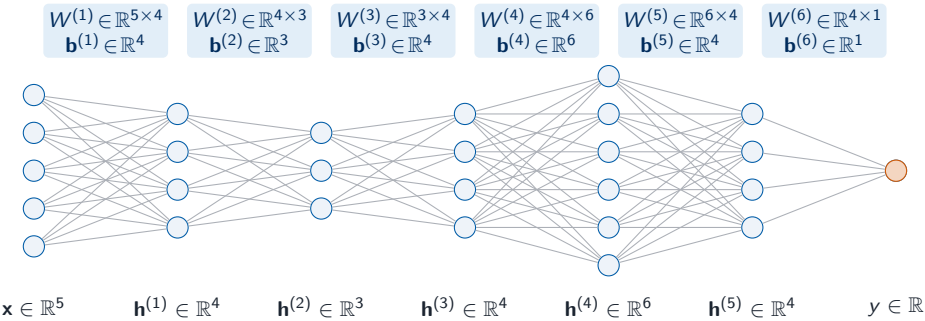
$$h_j^{(l)} = \sigma\Big(\sum_{i} h_i^{(l-1)}\, w_{i,j}^{(l)} + b_j^{(l)}\Big)$$

$$\iff \quad \mathbf{h}^{(l)} = \sigma\big(\mathbf{h}^{(l-1)} W^{(l)} + \mathbf{b}^{(l)}\big)$$

A network = nested matrix products and activations:

$$\mathbf{h}^{(2)} = \sigma\Big(\sigma\big(\mathbf{x} W^{(1)} + \mathbf{b}^{(1)}\big) W^{(2)} + \mathbf{b}^{(2)}\Big)$$

A Deep Network: $f : \mathbb{R}^5 \to \mathbb{R}$ with 5 Hidden Layers

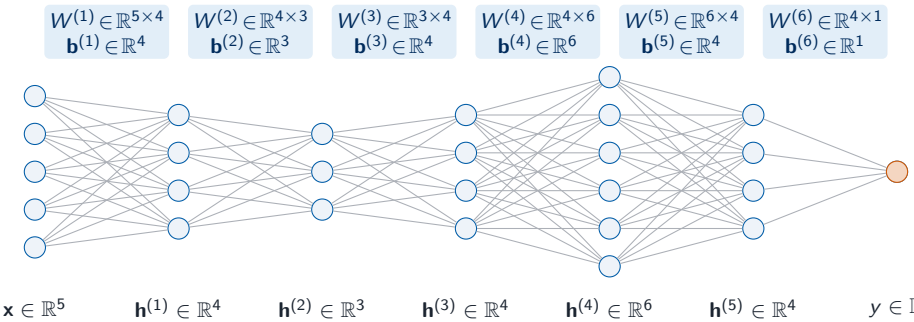


Layer by layer (every hidden layer: matrix product, bias, activation):

$$\mathbf{h}^{(1)} = \sigma\big(\mathbf{x}\,W^{(1)} + \mathbf{b}^{(1)}\big), \quad \mathbf{h}^{(2)} = \sigma\big(\mathbf{h}^{(1)}W^{(2)} + \mathbf{b}^{(2)}\big), \quad \mathbf{h}^{(3)} = \sigma\big(\mathbf{h}^{(2)}W^{(3)} + \mathbf{b}^{(3)}\big),$$
$$\mathbf{h}^{(4)} = \sigma\big(\mathbf{h}^{(3)}W^{(4)} + \mathbf{b}^{(4)}\big), \quad \mathbf{h}^{(5)} = \sigma\big(\mathbf{h}^{(4)}W^{(5)} + \mathbf{b}^{(5)}\big), \quad y = \mathbf{h}^{(5)}W^{(6)} + b^{(6)}$$

the output layer of a regression network is usually linear (no σ)

A Deep Network: $f : \mathbb{R}^5 \to \mathbb{R}$ with 5 Hidden Layers

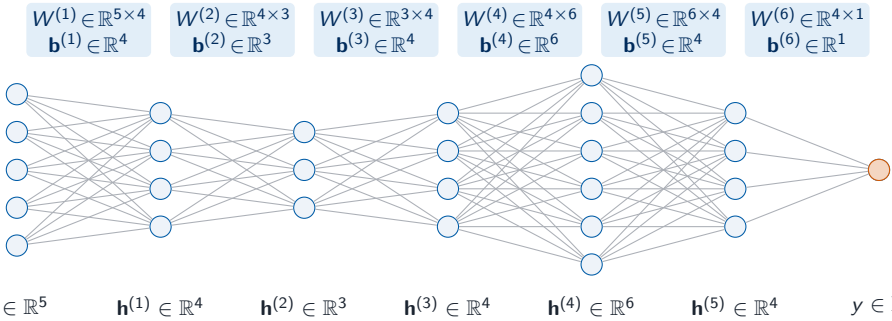


One formula — the argument of each layer is the previous layer:

$$f(\mathbf{x}) = \sigma\Big(\sigma\Big(\sigma\big(\sigma(\mathbf{x}\,W^{(1)} + \mathbf{b}^{(1)})\,W^{(2)} + \mathbf{b}^{(2)}\big)\,W^{(3)} + \mathbf{b}^{(3)}\Big)\,W^{(4)} + \mathbf{b}^{(4)}\Big)\,W^{(5)} + \mathbf{b}^{(5)}\Big)\,W^{(6)} + b^{(6)}$$

read from the inside out: the innermost bracket is the first layer

A Deep Network: $f : \mathbb{R}^5 \to \mathbb{R}$ with 5 Hidden Layers



As a composition of functions: $h^{(l)}(\mathbf{z}) = \sigma(\mathbf{z}\, W^{(l)} + \mathbf{b}^{(l)})$, $h^{(6)}(\mathbf{z}) = \mathbf{z}\, W^{(6)} + b^{(6)}$

$$f = h^{(6)} \circ h^{(5)} \circ h^{(4)} \circ h^{(3)} \circ h^{(2)} \circ h^{(1)}, \qquad f(\mathbf{x}) = h^{(6)}\Big(h^{(5)}\big(\cdots h^{(1)}(\mathbf{x})\cdots\big)\Big)$$

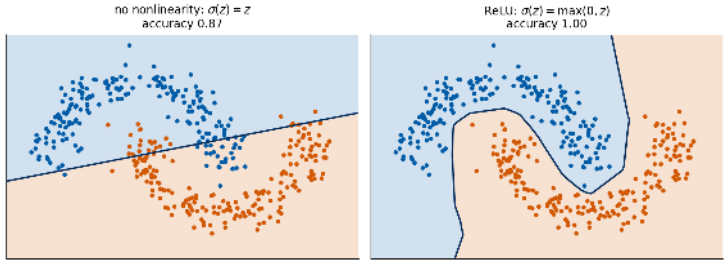
Parameters $\vec{\theta} = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{6}$: $24 + 15 + 16 + 30 + 28 + 5 = 118$ numbers.

Without a Nonlinearity, Depth Does Nothing

Two linear layers are one linear layer:

$$W_2\,(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{W_2 W_1}_{W}\,\mathbf{x} + \underbrace{W_2\mathbf{b}_1 + \mathbf{b}_2}_{\mathbf{b}}$$

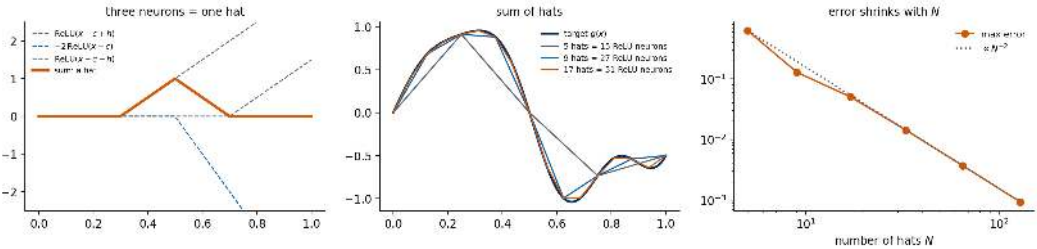
and by induction any stack of linear layers is a single affine map: a straight decision boundary.



the same network (3 hidden layers of 16 neurons) trained on two moons; accuracy 0.87 without a nonlinearity, 1.00 with ReLU

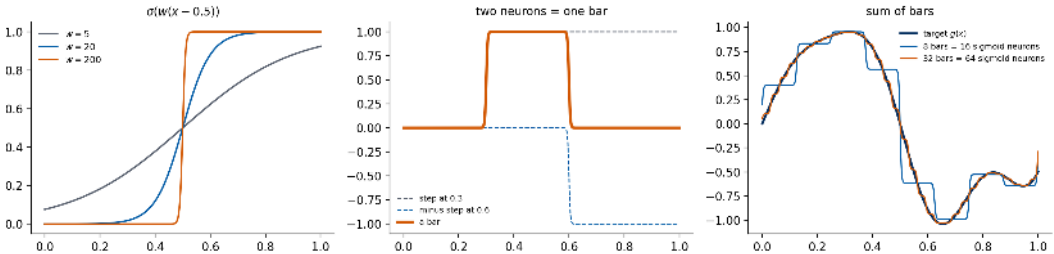
A Function Built from ReLU Pieces

Three ReLU neurons make a **hat** of width $2h$ at c:
$\Lambda_c(x) = \frac{1}{h}\left[\mathrm{ReLU}(x-c+h) - 2\,\mathrm{ReLU}(x-c) + \mathrm{ReLU}(x-c-h)\right]$



- hats on a grid c_k with heights $g(c_k)$: $\sum_k g(c_k)\,\Lambda_{c_k}(x)$ is the piecewise-linear interpolation of g, a one-hidden-layer network with weights written by hand
- the error falls as N^{-2} with the number of hats (bound $\frac{h^2}{8}\max|g''|$); measured slope -1.95

A Sigmoid Becomes a Step



- a large weight w turns $\sigma\big(w(x-b)\big)$ into a step at $x = b$
- two neurons: the difference of two steps is a **bar**; a sum of bars with heights g approximates g like a histogram
- this is the picture behind Cybenko's proof for sigmoidal activations

M. Nielsen, *Neural Networks and Deep Learning*, ch. 4: A visual proof that neural nets can compute any function

Universal Approximation Theorem

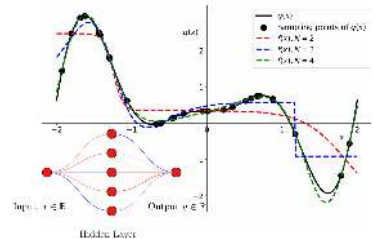
For any continuous g on a compact set $K \subset \mathbb{R}^d$ and any $\epsilon > 0$ there is a network with *one* hidden layer of N neurons,

$$f(\mathbf{x}) = \sum_{i=1}^{N} v_i\, \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i), \qquad \sup_{\mathbf{x} \in K} |f(\mathbf{x}) - g(\mathbf{x})| < \epsilon,$$

if and only if σ is **not a polynomial**.

It says: such weights exist.

It does not say: how many neurons (a grid of hats in d dimensions needs a number growing exponentially with d), nor how to find the weights: we train by minimising $\mathcal{L}(\vec{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \left|g(x_i) - f(x_i;\vec{\theta})\right|^2$.



more hidden neurons $N \to$ better approximation



examples of σ; any non-polynomial one works

G. Cybenko, Math. Control Signals Syst. **2**, 303 (1989), doi:10.1007/BF02551274; K. Hornik, M. Stinchcombe, H. White, Neural Netw. **2**, 359 (1989), doi:10.1016/0893-6080(89)90020-8; M. Leshno, V. Ya. Lin, A. Pinkus, S. Schocken, Neural Netw. **6**, 861 (1993), doi:10.1016/S0893-6080(05)80131-5

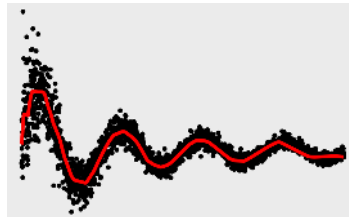
Deep Learning is a Non-Linear Curve Fitting

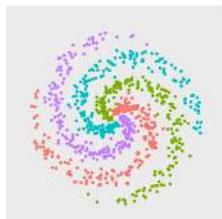
Any continuous function

$$f : R^N \rightarrow R^M$$

can be approximated by
NN with a single hidden layer









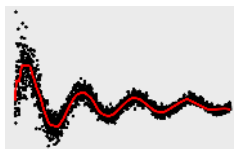
Problem Definition: Regression vs Classification

Regression

predict a **continuous** value

$$y \in \mathbb{R} \quad (\text{or } \vec{y} \in \mathbb{R}^n)$$

e.g. temperature, energy, concentration, binding affinity



output: a number; loss: e.g. mean squared error

Classification

predict a **discrete** label from a given set

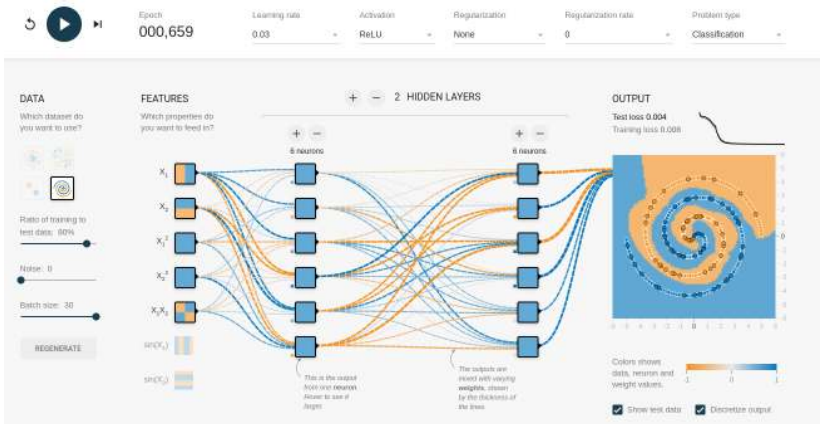
$$y \in \{c_1, c_2, \ldots, c_K\}$$

e.g. digit $0 \ldots 9$, cat or dog, ordered or disordered phase



output: probability per class; loss: e.g. cross-entropy

TensorFlow Playground



https://playground.tensorflow.org/

4

Literature

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Literature: Lecture Notes, ML for Physics and Science

arXiv > quant-ph > arXiv:2508.10666

Deep Learning in Classical and Quantum Physics

Dedicated lecture notes · T. Heightman, M. Płodzień

arXiv:2508.10666



Modern applications of machine learning in quantum sciences

Machine learning in quantum sciences · A. Dawid et al.

arXiv:2204.04198

A high-bias, low-variance introduction to Machine Learning for physicists

A high-bias, low-variance introduction to Machine Learning for physicists

Phys. Rep. 810, 1–124 · P. Mehta et al.

doi:10.1016/j.physrep.2019.03.001

arXiv > physics > arXiv:2102.04883

Introduction to Machine Learning for the Sciences

T. Neupert, M. H. Fischer, E. Greplova, K. Choo, M. M. Denner

arXiv:2102.04883

nature

Scientific discovery in the age of artificial intelligence

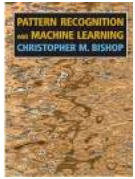
Nature 620, 47–60 · H. Wang et al.

nature reviews physics

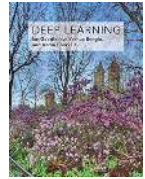
On scientific understanding with artificial intelligence

Nature Reviews Physics 4, 761–769 · M. Krenn et al.

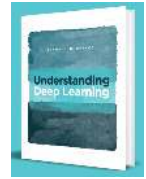
Literature: Mathematics and Broader Context of AI



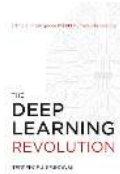
Pattern Recognition and Machine Learning
C. M. Bishop



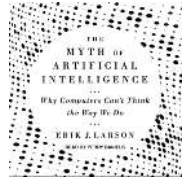
Deep Learning
I. Goodfellow, Y. Bengio, A. Courville



Understanding Deep Learning
S. J. D. Prince



The Deep Learning Revolution
T. J. Sejnowski



The Myth of Artificial Intelligence: Why Computers Can't Think the Way We Do
E. J. Larson

Summary

1. **Landscape of AI research**; the main paradigm today is *data-driven*
2. **Deep learning is a subset of machine learning**: machine learning uses hand-crafted features, deep learning extracts the features automatically
3. **Neural networks are parametrised non-linear functions with a nested structure**
4. **Nonlinear activations are essential**: without them any depth collapses to one linear map
5. **Universal approximation theorem**: one hidden layer with a non-polynomial activation approximates any continuous function; it guarantees existence, not size or training
6. **Deep learning is non-linear curve fitting**: regression and classification

Next Lecture

1. Elements of deep learning: loss functions, gradient descent, mini-batches
2. Why neural networks work so well: learning a probability distribution from samples
3. Deep learning "Hello world": MNIST

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar

